

RELAZIONE TECNICA HOMEWORK 5 – Ingegneria dei dati 2025/2026

Sistema avanzato di search su articoli scientifici, in cui le tabelle e le figure siano trattate come oggetti di prima classe e completamente indicizzabili.

1. INTRODUZIONE

1.1 Obiettivi del progetto

Questo progetto si propone di sviluppare un sistema avanzato di information retrieval per articoli scientifici, in cui tabelle e figure siano trattate come oggetti di prima classe e completamente indicizzabili. Il sistema è stato implementato in due varianti:

- Progetto 5: ricerca su articoli di arXiv relativi a “query processing” OR “query optimization”
- Progetto 5.1: ricerca su articoli di PubMed relativi a “cancer risk” AND “coffee consumption”

Gli obiettivi specifici includono:

- Acquisizione del corpus: scraping di articoli scientifici da repository pubbliche
- Indicizzazione: creazione di indici Lucene per ricerca avanzata
- Estrazione: identificazione e parsing di tabelle e figure
- Indicizzazione avanzata: indicizzazione separata di tabelle e figure come entità indipendenti
- interfacce di ricerca multiple: CLI e Web UI per ricerche su documenti, tabelle e figure
- Valutazione prestazionale: test quantitativi e qualitativi del sistema

1.2 Motivazioni scientifiche

Le tabelle e le figure negli articoli scientifici contengono informazioni dense e strutturate spesso difficili da recuperare con ricerche testuali tradizionali. Un sistema che permetta di ricercare direttamente in questi elementi consente:

- Maggiore precisione nel recupero di dati sperimentali
- Identificazione rapida di risultati quantitativi
- Confronto cross-paper di metodologie e risultati
- Analisi semantica del contesto delle evidenze scientifiche

2. ARCHITETTURA DEL SISTEMA

2.1 Architettura generale

Il sistema segue un'architettura a pipeline modulare composta da quattro fasi principali:

1. Scraping: acquisizione del corpus dal web
2. Indexing: indicizzazione con Lucene
3. Extraction: estrazione di tabelle e figure
4. Searching: interfacce CLI e web UI per fare query

Ogni fase è indipendente e riutilizzabile, permettendone la modifica singolare senza compromettere il resto della pipeline.

2.2 Stack tecnologico

Il codice è scritto in Java tramite maven:

- Sviluppo applicativo: Java 17
- Gestione dipendenze: Maven 3.1.0
- Indicizzazione e ricerca: Apache Lucene 9.11.1
- Parsing documenti html: Jsoup 1.16.1
- Web scraping: OkHttp 4.12.0
- Parsing feed Atom (arXiv): ROME 1.18.0
- Serializzazione dati: Jackson 2.17.1
- Interfaccia web: Jetty 12.0.7

2.3 Struttura dei moduli

Entrambi i progetti sono organizzati in quattro moduli principali

2.3.1 Scraper

Questo è l'unico modulo in cui c'è un file specifico e unico per ognuno dei due progetti.

- ArxivScraper.java: scraping articoli da arXiv, unico per il progetto 5
- PubMedScraper.java: scraping articoli da PubMed, unico per il progetto 5.1
- Main.java: punti di ingresso per l'acquisizione del corpus

2.3.2 Indexer

- Indexer.java: indicizzazione documenti html
- TableIndexer.java: indicizzazione tabelle estratte
- FigureIndexer.java: indicizzazione figure estratte

2.3.3 Extractor

- TableExtractor.java: estrazione delle tabelle dai documenti
- FigureExtractor.java: estrazione delle figure dai documenti

2.3.4 Searcher

- Searcher.java: CLI per ricerca documenti
- TableSearcher.java: CLI per ricerca tabelle
- FigureSearcher.java: CLI per ricerca figure
- WebSearcher.java: web UI per ricerca documenti
- WebTableSearcher.java: web UI per ricerca tabelle
- WebFigureSearcher.java: web UI per ricerca figure

3. IMPLEMENTAZIONE TECNICA

3.1 Scraping e acquisizione corpus

3.1.1 Scraping da arXiv (progetto 5)

L'API di arXiv fornisce un endpoint Atom feed che permette query strutturate

```
```java
```

```
String searchQuery = "ti:\"Query processing\" OR abs:\"Query processing\" " +
```

```
"OR ti:\"Query optimization\" OR abs:\"Query optimization\"";
```

```
...
```

Flusso di esecuzione:

1. Richiesta http GET all'API con query string OR string
2. Parsing del feed Atom con libreria ROME
3. Estrazione ID arXiv per ogni articolo trovato
4. Download dell'html completo da ar5iv.org (versione html renderizzata da LaTeX)
5. Salvataggio dei file in '/corpus-html'

Sfide tecniche

- Rate limit: arXiv limita a 3 richieste al secondo, implementato throttling con 'Thread.sleep(350)'
- Conversioni incomplete: alcuni LaTeX derivati da html generano errori, è stata implementata una validazione con controllo
- Paginazione: risultati limitati a 100 per richiesta, gestione offset con loop

Risultati:

- Corpus di 1125 articoli html
- Dimensione corpus: 749 MB

### 3.1.2 Scraping da PubMed (progetto 5.1)

PubMed offre E-utilities per ricerca programmatica:

```
```java
```

```
// Fase 1: Ricerca ID tramite esearch
```

```
String ESEARCH_URL = "https://eutils.ncbi.nlm.nih.gov/entrez/eutils/esearch.fcgi";
```

```
// Query: db=pmc&term=cancer+risk+AND+coffee+consumption&retmax=500
```

```
// Fase 2: Download HTML tramite PMC ID
```

```
String PMC_HTML_URL = "https://www.ncbi.nlm.nih.gov/pmc/articles/PMCxxxxxxx/";
```

```
...
```

Flusso di esecuzione:

1. Query su database PMC con filtro open access
2. Parsing risposta XML con Jsoup per estrarre i PMC ID
3. Download del corrispettivo html per ogni PMC ID
4. Salvataggio in '/corpus-html'

Sfide tecniche;

- Rate limit: limita a 3 richieste al secondo, implementato throttling con 'Thread.sleep(350)'
- Accesso limitato: solo articoli open access, applicato tramite un filtro nella query
- Formato vario: utilizzano un html non standardizzato

Risultati:

- Corpus di 500 articoli, numero scelto arbitrariamente
- Tutti articoli recenti (2020-2025)

3.2 Indicizzazione

3.2.1 Schema di indicizzazione

Ogni documento viene indicizzato con i seguenti campi

Campo	Tipo	Analyzer	Stored	Descrizione
fileName	StringField	-	Yes	Nome del file
title	TextField	Custom	Yes	Titolo articolo
authors	TextField	Custom	Yes	Lista autori
date	StringField	-	Yes	Data pubbl.
abstract	TextField	English	Yes	Abstract
fulltext	TextField	English	Yes	Testo completo

I campi senza analyzer sono stati indicizzati così come sono, in modo da avere un match esatto nelle ricerche.

L'English analyzer è un analyzer standard per la lingua inglese, efficace sui lunghi testi in lingua inglese, applica le comuni tecniche come stemming e rimozione stopwords.

Per gli altri campi che non sono lunghi testi, e per cui non si vuole un match esatto, sono stati generati degli analyzer custom. Su title si usa una tokenizzazione su spazi, conversione in minuscolo e divisione di parole in base a maiuscole, numeri, simboli, ecc... Su authors si usa una tokenizzazione su spazi e conversione in minuscolo.

3.2.2 Parsing html e estrazione campi

Per il parsing è stato usato Jsoup, che permette di navigare il DOM html:

```
```java
```

```
Document htmlDoc = Jsoup.parse(htmlContent);
```

```
htmlDoc.select("script, style").remove(); // Rimozione elementi non testuali
```

```
// Estrazione titolo
```

```
String title = htmlDoc.title();
```

```
if (title.isEmpty()) {
```

```
 title = htmlDoc.selectFirst("h1.title").text();
```

```
}
```

```
// Estrazione autori
```

```
List<String> authors = new ArrayList<>();
```

```
for (Element a : htmlDoc.select("div.authors a")) {
```

```

 authors.add(a.text());
}

// Estrazione abstract
String abstractText = htmlDoc.select("div.ltx_abstract").text();

// Estrazione full-text
String fullText = htmlDoc.text();
...

```

Nel caso in cui la conversione in html generi un errore, il documento viene saltato. Sono gestiti encoding UTF-8 e ISO-8859-1. In caso di campi mancanti, vengono inserite delle stringhe vuote.

### 3.2.3 Configurazione IndexWriter

```

```java
Analyzer analyzer = new PerFieldAnalyzerWrapper(analyzerEN, analyzerPerField);
Directory directory = FSDirectory.open(indexDir);
IndexWriterConfig config = new IndexWriterConfig(analyzer);
IndexWriter writer = new IndexWriter(directory, config);

// Indicizzazione con update semantics
writer.updateDocument(new Term("filename", fileName), doc);
...

```

Come chiave viene usato il fileName.

3.3 Estrazione tabelle e figure

3.3.1 Estrazione tabelle

Algoritmo implementato:

1. Identificazione delle tabelle tramite selezione di elementi dal DOM (<table>)
2. Estrazione delle caption tramite ricerca di elementi associati (<caption>, <figcaption>)
3. Parsing del body tramite conversione delle celle in testo
4. Identificazione menzioni tramite ricerca di paragrafi che menzionano il nome della tabella
5. Costruzione del contesto selezionando i paragrafi contenenti termini della tabella

```

```java
// Pseudo-codice semplificato
for (Element table : doc.select("table")) {
 String tableId = table.attr("id");
 String caption = table.select("caption, figcaption").text();
}

```

```

String body = extractTableText(table);

// Estrazione mentions
List<String> mentions = new ArrayList<>();
for (String paragraph : allParagraphs) {
 if (paragraph.contains(tableId) || paragraph.contains("Table")) {
 mentions.add(paragraph);
 }
}

// Estrazione context
Set<String> keywords = extractKeywords(caption + " " + body);
List<String> contextParagraphs = findContextParagraphs(allParagraphs,
keywords);

ExtractedTable table = new ExtractedTable(paperId, tableId, caption,
body, mentions, contextParagraphs);

results.add(table);
}
...

```

In output restituisce un file json per ogni paper (paperId\_tables.json), ed ognuno di questi file contiene un array di oggetti tabella, con le relative proprietà.

### 3.3.2 Estrazione figure

Per le figure è stato scritto un algoritmo analogo

1. Identificazione delle figure tramite tag (<figure>,<img>)
2. Estrazione dell'URL dell'immagine
3. Estrazione della caption dell'immagine
4. Identificazione delle menzioni nei paragrafi
5. Non è presente il contesto poiché le figure non hanno un body testuale

```

6. ```java
7. for (Element figure : doc.select("figure, img")) {
8. String figureId = figure.attr("id");
9. String url = extractImageUrl(figure);
10. String caption = figure.select("figcaption").text();
11.
12. List<String> mentions = findMentions(allParagraphs, figureId);
13.
14. ExtractedFigure fig = new ExtractedFigure(paperId, figureId,
15. url, caption, mentions);
16. results.add(fig);
17. }
18. ```

```

## 3.4 Indicizzazione tabelle e figure

### 3.4.1 Schema indice tabelle

<b>Campo</b>	<b>Tipo</b>	<b>Analyzer</b>	<b>Descrizione</b>
paper_id	TextField	Whitespace	Id del paper
table_id	TextField	Whitespace	Id della tabella
caption	TextField	English	Caption della tabella
body	TextField	Whitespace	Contenuto tabella
mentions	TextField	English	Paragrafi che citano la tabella
context_paragraphs	TextField	English	Paragrafi di contesto

Per il body è stato usato solo il whitespace per separare le parole essendo informazioni di tipo tecnico. Anche per gli id si è scelto di usare solo il whitespace, volendo un match esatto nella ricerca. Per gli altri campi testuali si è scelto l'analyzer inglese standard.

### 3.4.2 Schema indice figure

<b>Campo</b>	<b>Tipo</b>	<b>Analyzer</b>	<b>Descrizione</b>
paper_id	TextField	Whitespace	Id del paper
figure_id	TextField	Whitespace	Id della figura
url	TextField	Whitespace	url della figura
caption	TextField	English	Caption della figura
mentions	TextField	English	Paragrafi che citano la figura

La scelta degli analyzer segue le stesse motivazione delle tabelle.

### 3.4.3 Processo di indicizzazione tabelle e figure

```

` `` ` java

// Lettura JSON con Jackson

ObjectMapper mapper = new ObjectMapper();

List<ExtractedTable> tables = mapper.readValue(jsonFile,

 new TypeReference<List<ExtractedTable>>() {});

// Indicizzazione in Lucene

for (ExtractedTable table : tables) {

 Document doc = new Document();

 doc.add(new TextField("paper_id", table.paperId, Field.Store.YES));

 doc.add(new TextField("table_id", table.tableId, Field.Store.YES));

 doc.add(new TextField("caption", table.caption, Field.Store.YES));

 doc.add(new TextField("body", table.body, Field.Store.YES));

 doc.add(new TextField("mentions", String.join("\n", table.mentions),

 Field.Store.YES));

```

```

doc.add(new TextField("context_paragraphs",
 String.join("\n", table.contextParagraphs),
 Field.Store.YES));

writer.addDocument(doc);
}
...

```

Risultati:

- Progetto 5 (arXiv): 2986 tabelle indicizzate e 14898 figure indicizzate
- Progetto 5.1 (PubMed): 1542 tabelle indicizzate e 1817 figure indicizzate

## 3.5 Ricerca

### 3.5.1 Ricerca CLI

Per la ricerca tramite CLI è stato implementato un input interattivo da console che inizialmente spiega con brevi istruzioni come scrivere le query, e poi consente di fare query sequenziali finché non si decide di uscire.

Le query seguono la sintassi Lucene permettendo la ricerca multi-campo booleana, di parole o frasi esatte.

Il ranking dei documenti risultanti dalla query è deciso tramite BM25, che è la scelta di default di Lucene.

L'output della ricerca è visibile in console, con i risultati numerici e poi degli esempi dei documenti trovati, non tutti per non intasare la vista del terminale.

### 3.5.2 Ricerca web

Per la ricerca tramite interfaccia web, sono state usate le librerie jetty e jakarta.

Il funzionamento è uguale alla ricerca tramite CLI, ma le informazioni appaiono su una pagina web e la ricerca è inserita all'interno di una form con un singolo campo per la query.

## 4. ESPERIMENTI E VALUTAZIONE

### 4.1 Metriche quantitative

Metrica	Progetto 5 (arXiv)	Progetto 5.1 (PubMed)
Documenti scaricati	1125	500
Tabelle estratte	2986	1542
Figure estratte	14898	1817
Dimensione corpus html	749 MB	153 MB
Dimensione indice docs	53 MB	48 MB
Dimensione indice tabelle	122 MB	68 MB
Dimensione indice figure	7 MB	13 MB



L'operazione di scraping è stata l'unica a richiedere un significativo lasso di tempo, per cui prenderemo quella come misurazione quantitativa. Lo scraping per arXiv ha impiegato 9 minuti circa, mentre per PubMed circa 6 minuti. Considerando che per arXiv sono stati scaricati più del doppio dei documenti, ci ha impiegato meno del doppio del tempo.

Le dimensioni dell'indice di arXiv sono molto ridotte rispetto alla dimensione dei corpus, circa il 7%. Mentre per PubMed siamo circa al 30% della dimensione del corpus, quindi molto meno efficiente.

## 4.2 Valutazione qualitativa

Il sistema è stato testato con diverse tipologie di query:

- Tramite singole parole, che in base alla specificità della parola possono produrre più o meno risultati rilevanti
- Tramite frasi, che migliorano la precisione filtrando documenti più specifici
- Tramite query booleane che permettono di ricercare più tipologie di documenti con una singola query (OR) e quindi essere più lasche, oppure essere più specifiche delle parole singole ma meno delle frasi (AND)

Il risultato dipende anche dai campi sui quali si effettua la ricerca. Utilizzando campi testuali ampi, analizzati tramite l'english analyzer standard, si può gestire ampiamente il margine di specificità voluto. Mentre per campi più brevi e meno elaborato dell'indicizzazione, come gli autori o il contenuto delle tabelle, c'è la necessità di essere molto precisi per ottenere i documenti cercati, questo è voluto poiché questi campi contengono informazioni molto specifiche che solitamente non vengono cercate a maglia larga. Infine, per gli identificativi c'è il bisogno di fare un match esatto.

### 4.2.1 Punti di forza

Gli analyzer possono essere modificati a piacere per ottenere i risultati desiderati.

Il sistema è molto modulare, ogni fase è indipendente dalle altre e può essere modificata agevolmente.

L'indicizzazione e soprattutto la ricerca sono molto veloci, almeno sul numero di documenti utilizzato in questo homework.

Per utenti meno esperti nell'informatica, c'è l'interfaccia via web che permette un utilizzo estremamente facile del sistema di ricerca.

### 4.2.2 Limitazioni osservate

Alcune tabelle complesse perdono la struttura durante l'estrazione.

Le formule matematiche in LaTeX non vengono sempre convertite bene in testo.

Nel testo in generale, e più nello specifico nella captions, alcuni termini tecnici possono generare rumore.

### 4.2.3 Confronto tra i domini

Gli articoli di arXiv sono molto tecnici con notazioni matematiche, tabelle di benchmark di metriche, elevato numero di figure con grafici e diagrammi architetture, elevata presenza di terminologia specialistica

Gli articoli di PubMed sono orientati a evidenze cliniche, per cui le tabelle, le figure e i termini, per quanto anch'essi di ambito specialistico, sono molto più omogenei.

ArXiv beneficia di analyzer che preservano termini tecnici, soprattutto nelle tabelle, mentre PubMed richiede una gestione frequente di acronimi medici.

#### **4.2.4 Scalabilità**

Lucene è in grado di operare su milioni di documenti, ma con tecniche più avanzate rispetto a quelle utilizzate in questo progetto.

Ad esempio, per lo scraping che è la fase che ha richiesto più tempo, si ha la forte limitazione delle 3 richieste al secondo, che potrebbe essere gestita tramite parallelizzazione con thread pool.

Lo stesso discorso è valido per il parsing, che potrebbe essere velocizzato tramite elaborazione parallela.

Per lo storage, gli indici sono di dimensioni gestibili, è il corpus dei documenti che richiede molto spazio. Ad esempio, se per circa 1000 articoli, il corpus pesa circa 750 MB, e l'indice circa 50 MB; per un milione di documenti, il corpus occuperebbe 750 GB e l'indice 50 GB circa. Quindi in realtà sarebbe comunque ben gestibile.

## **6. CONCLUSIONI**

Il progetto ha dimostrato la fattibilità e l'efficacia di un sistema di information retrieval che tratta tabelle e figure come entità di prima classe.

Con la ricerca sul testo completo degli articoli diventa complesso andare ad estrarre informazioni tecniche precise e in breve tempo, anzi può succedere che non tutti i valori inseriti nelle tabelle e nelle figure vengano poi menzionati nel testo.

Mentre avendo tabelle e figure su cui cercare singolarmente, si semplifica di molto l'estrazione di informazioni tecniche e precise, senza rumore indesiderato, aumentando quindi sia precision che recall. Questo può essere utile per identificare rapidamente le evidenze empiriche, per aggregare risultati sperimentali da più fonti, per fare confronti tra articoli diversi, per monitorare l'evoluzione delle metriche nel tempo, e per molti altri scopi.

Di pari passo però, diventa molto più critico avere un sistema efficace nell'estrazione e nell'indicizzazione di tabelle e figure.